

```
// no 7
let name = "Tracy";
let age = 20;

console.log(
  "My name is " + name +
  " and I am " + age +
  " years old."
);

// no 8
let school = "WITU";
console.log(school);

const country = "Uganda";
console.log(country);

var city = "Kampala";
console.log(city);

// no 9
let text = "JavaScript";
let number = 10;
let isStudent = true;

console.log(text);
console.log(number);
```

```
console.log(isStudent);

// no 10
let name = "John";
let age = 22;
let color = "Blue";

console.log(
  "My name is " + name +
  ", I am " + age +
  " years old and my favorite color is " + color
);

// no 11
let num1 = 10;
let num2 = 5;

console.log(num1 + num2);
console.log(num1 - num2);
console.log(num1 * num2);
console.log(num1 / num2);
console.log(num1 % num2);

// no 12
let a = 15;
```

```
let b = 5;

console.log(a + b);
console.log(a - b);
console.log(a * b);
console.log(a / b);

// no 13
let firstName = "Jonathan";
let lastName = "Grace";

let fullName =
firstName + " " + lastName;

console.log(fullName);

// no 14.
let x = 10;
let y = "10";

console.log(x == y);

// true because == compares values and ignores data
types

// no 15
// 15. What is the output?
```

```
let a = 5;
let b = 2;

console.log (a + b);

// output is 7 because + operator adds the two
numbers together

// no 16

// 16.What is the output?
let num1 = 4;
let num2 = 6;
console.log (num1 * num2);

// output is 24 because * operator multiplies the
two numbers together

// no 17

// 17. Predict the result:

console.log (5 === "5");
```

```
// output is false because === operator compares  
both value and data type, and here we have a number  
and a string which are not the same data type
```

```
// no 18
```

```
// 18.What will this print?
```

```
// i) console.log (typeof "Hello");
```

```
// output is "string"
```

```
// ii) console.log (typeof 50);
```

```
// output is "number"
```

```
// iii) console.log (typeof true);
```

```
// output is "boolean"
```

```
// iv) console.log (typeof "100");
```

```
// output is "string"
```

```
// v) let x = 20;
```

```
// x = 30;
```

```
// console.log(x);
```

```
// output is 30 because we have reassigned the
value of x to 30

// vi) const age = 18;
// age = 25;
// console.log(age);
// output is error because we cannot reassign a
value to a constant variable

// vii) let name = "John";
// console.log (name + " Doe")
// output is "John Doe" because the + operator
concatenates the two strings together

// viii) let x = 5;
// let y = 10;
// console.log (x + y * 2);
// output is 25 because of operator precedence, the
multiplication is performed before the addition, so
it becomes 5 + (10 * 2) which equals 5 + 20 = 25

// ix) let x = "5";
// let y = 5;
// console.log (x + y);
```

```
// output is "55" because the + operator
concatenates the string "5" with the number 5,
resulting in the string "55"

// x) console.log (10 % 3);
// output is 1 because the % operator returns the
remainder of the division of 10 by 3, which is 1

// xi) console.log (8 > 3);
// output is true because 8 is greater than 3

// xii) console.log (5 < 2);
// output is false because 5 is not less than 2

// xiii) console.log (7 >= 7);
// output is true because 7 is equal to 7, and the
>= operator returns true if the left operand is
greater than or equal to the right operand

// xiv) console.log (4 != 6);
// output is true because 4 is not equal to 6

// no 19
// 19. Identify the data type of:
// i) "JavaScript"
// string
```

```
// ii) 100
// number

// iii) false
// boolean

// iv) true
// boolean

// v) "100"
// string

// vi) 25
// number

// vii) false
// boolean

// no 20

// 20. Explain the difference between, with code
snippets:
// i) let
// let is a keyword used to declare a variable that
can be reassigned a new value later in the code. It
```


is block-scoped, meaning that the variable is only accessible within the block it is declared in.

```
// let x = 5;
```

```
// ii) const
```

// const is a keyword used to declare a variable that cannot be reassigned a new value after it has been initialized. It is also block-scoped, meaning that the variable is only accessible within the block it is declared in.

```
// const y = 10;
```

```
// iii) var
```

// var is a keyword used to declare a variable that can be reassigned a new value later in the code. It is function-scoped, meaning that the variable is accessible throughout the entire function it is declared in, regardless of block scope.

```
// var z = 15;
```

```
// n0 21
```

```
// 21. Explain the difference between, with code snippets:
```

```
// i) ==
```

```
// The == operator is used for loose equality comparison. It compares the values of the operands and performs type coercion if necessary, meaning it converts the operands to a common type before making the comparison.
```

```
// console.log(5 == "5"); // true because the string "5" is coerced to the number 5 before comparison
```

```
// ii) ===
```

```
// The === operator is used for strict equality comparison. It compares both the values and the data types of the operands, without performing type coercion.
```

```
// console.log(5 === "5"); // false because the number 5 and the string "5" are different data types
```

```
// no 22
```

```
// 22.Differentiate between variables and data types with examples.
```

```
// A variable is a container used to store data.
```

```
// let name = "Alice";  
// Data types are the types of data that a variable  
can hold, such as numbers, strings, booleans, etc.  
// let age = 25; // number  
// let isActive = true; // boolean
```

```
// no 23
```

```
// 23.What is a data type? (2 Marks)  
// A data type is a classification of data that  
tells the computer how to interpret and manipulate  
the data. It defines the kind of value a variable  
can hold, such as numbers, strings, booleans, etc.  
Data types help ensure that operations performed on  
the data are appropriate for its type.
```

```
// no 24
```

```
// 24.Differentiate between JavaScript variables  
and operators, with examples
```

```
// JavaScript variables are used to store data  
values. They can hold different types of data, such  
as numbers, strings, booleans, etc.
```

```
let name = "Bob"; // variable storing a string  
value
```

```
// JavaScript operators are symbols that perform
operations on variables and values. They can be
arithmetic, comparison, logical, etc.
let x = 10;
let y = 5;
console.log(x + y); // using the + operator to add
two variables, output is 15
```

```
// no 26
```

```
// 26.Which of these is a valid variable name?
// i) 2name
// ii) myName is a valid variable name because it
starts with a letter and does not contain any
spaces or special characters.
// iii) let
// iv) let user-name = "John";
// v) let userName = "John"; is a valid variable
name because it starts with a letter and uses
camelCase notation without any spaces or special
characters.
```

```
// no 27
```

```
// // 27. Complete the code:
// _____ score = 90;
```

```
// console.log(score);
// let score = 90;

// no 28
    // 28. let number = ____;
    // Assign the value 50

let number = 50;


// no 29
// 29. Identify and correct the error:
// i) let age = 20
// console.log (age
// The error in this code is that the console.log
statement is missing a closing parenthesis. The
corrected code should be:
let age = 20;
console.log(age);


// ii) let city Kampala;
// console.log(city);
// The error in this code is that the variable
declaration is missing an assignment operator (=)
to assign a value to the variable. The corrected
code should be:
```

```
let city = "Kampala";
console.log(city);

// iii) var score = 90;
// console.log Score;
// The error in this code is that the variable name
// is case-sensitive and should be written as "score"
// instead of "Score". The corrected code should be:
var score = 90;
console.log(score);

// iv) cosole.log("Welcome");
// The error in this code is a typo in the
// console.log function. The corrected code should be:
console.log("Welcome");

// v) const name = "John"; name = "Peter";
// The error in this code is that a constant
// variable cannot be reassigned a new value. The
// corrected code should be:
const name = "John";
// name = "Peter"; // This line should be removed
// or commented out to avoid the error
```